Reviewer Pack — Minimal Number of Representations as Maximal Ideals in Algeb…

Entry c0b307822ab7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 23:39:21 UTC

Minimal Number of Representations as Maximal Ideals in Algebraic Geometry vs. Resolution Proof Width

Entry ID: `c0b307822ab7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 23:39:21 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Ideal Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every n-variable CNF φ, the number of representations of its associated Tseitin polynomial as a maximal ideal in the ring of polynomials modulo φ is upper-bounded by the resolution proof width w(φ), i.e., #M(φ) ≤ w(φ).

Rationale (proposer's reasoning):

> Algebraic geometry provides a rich framework for studying ideals, which can be used to represent computational structures. Maximal ideals correspond to prime elements in polynomial rings, and their count may relate to the complexity of resolving logical statements. This conjecture explores the potential link between ideal theory and resolution proof complexity.

Taxonomy category: `IdealTheoreticResolution` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f8683507081170d3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if and only if for every n-variable CNF φ, the number of representations as a maximal ideal in the ring modulo φ (#M(φ)) is less than or equal to the resolution proof width (w(φ)), with no seed producing #M(φ) > w(φ).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Tseitin polynomial" AND "maximal ideal" IN TITLE - "resolution proof complexity" AND "algebraic geometry" IN TITLE - "#M(φ) ≤ w(φ)" AND ("ideal theory" OR "algebraic geometry")

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def generate_cnf(n):
    cnf = []
    for _ in range(10):  # Generate 10 clauses for simplicity
        clause = [random.randint(-n, n) for _ in range(3)]
        if all(x != 0 for x in clause):
            cnf.append(clause)
    return cnf

def tseitin_polynomial(cnf):
    literals = set()
    for clause in cnf:
        literals.update(abs(lit) for lit in clause)

    clauses = []
    new_vars = {}
    var_count = 1

    for i, literal in enumerate(literals):
        new_var = f"v{var_count}"
        new_vars[literal] = new_var
        new_vars[-literal] = f"~{new_var}"
        var_count += 1

    for clause in cnf:
        disjunction = []
        for lit in clause:
            if lit > 0:
                disjunction.append(new_vars[lit])
            else:
                disjunction.append(f"~{new_vars[-lit]}")
```

```
        conjunction = []
        for i, lit in enumerate(clause):
            negated_lit = f"~{new_vars[lit]}"
            conjunction.extend([negated_lit] + [f"v{j+1}" for j in range(i) if j != i])

        clauses.append(conjunction)

    return clauses

def resolution_width(cnf):
    def resolve(clause1, clause2):
        resolved = []
        for lit1 in clause1:
            for lit2 in clause2:
                if lit1 == -lit2:
                    continue
                resolved.append(lit1)
        return resolved

    queue = cnf[:]
    while True:
        new_clauses = []
        found_resolvent = False

        for i in range(len(queue)):
            for j in range(i + 1, len(queue)):
                resolvent = resolve(queue[i], queue[j])
                if not resolvent:
                    continue
                found_resolvent = True
                new_clauses.append(resolvent)

        if not found_resolvent:
            break

        queue.extend(new_clauses)

    return max(len(clause) for clause in queue)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 5 + (seed % 6) * 5  # Sweep through sizes 5, 10, 15, 20, 30, 40
    cnf = generate_cnf(n)
    tseitin_poly = tseitin_polynomial(cnf)
    width = resolution_width(tseitin_poly)

    return {
        "metric_name": "#M(φ)",
        "metric_value": len(tseitin_poly),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": len(tseitin_poly) <= width,
        "counterexample": "" if len(tseitin_poly) <= width else f"Counterexample for n={n}"
    }
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0cfd85da.py", line 110, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0cfd85da.py", line 93, in run_trial
    width = resolution_width(tseitin_poly)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0cfd85da.py", line 75, in resolution_wi
    resolvent = resolve(queue[i], queue[j])
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0cfd85da.py", line 63, in resolve
    if lit1 == -lit2:
               ^^^^^
TypeError: bad operand type for unary -: 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot verify if the conjecture is supported or falsified. | next: Re-run the test with appropriate error handling to ensure it completes without crashing and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16198
2	preregistration	ollama_remote	glm4:latest	0	0	10522
3	novelty	ollama_remote	glm4:latest	0	0	8958
4	novelty	ollama_remote	glm4:latest	0	0	16181
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35417
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	49101
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13614
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16969
9	judge	ollama_remote	glm4:latest	0	0	9265

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 176227 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c0b307822ab7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c0b307822ab7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c0b307822ab7.tar.gz` (if generated)